Tech Intern Assignment

Breathe ESG

Context

Breathe ESG ingests emissions and activity data from client companies. The hard part of our job isn't computing carbon, it's that every client's data lives somewhere different, in a different shape, with different gaps. SAP exports for fuel and procurement. Utility bills as PDFs or portal scrapes. Travel data from Concur or similar. Spreadsheets a sustainability lead maintains by hand.

A PM sends you this:

> *"We're onboarding a new enterprise client. They've got fuel and procurement data sitting in SAP, electricity data their facilities team pulls from utility portals, and business travel data from a corporate travel platform. We need to ingest all of it, normalize it, and let our analysts review and sign off before it goes to auditors. Build a prototype in Django and React. You have 4 days."*

You may use any AI tool. We expect you to. **We are not evaluating typing speed. We are evaluating judgment.**

A note on quality

We don't want slop. We've all seen what unchecked AI output looks like: generic CRUD apps, READMEs written like marketing copy, code that works but no one understood before it was committed, "best practices" copy-pasted with no thought to whether they fit the problem.

Submit less, but submit work you understand and can defend. A smaller app with a sharp data model and honest tradeoffs will beat a feature-rich app you can't explain. If we ask "why did you do X" and the answer is "the AI suggested it," that's a fail regardless of whether X was correct.

What to build

A Django REST and React app that ingests data from three source *types*, normalizes it, and surfaces a review dashboard where an analyst can see what came in, what failed, what looks suspicious, and approve rows before they're locked for audit.

We are not providing mock APIs or sample files. Part of the assignment is figuring out what these data sources actually look like in the real world. Research SAP export formats. Look at what a utility data export typically contains. Read the Concur or Navan API docs. Then design something that handles realistic shapes, not toy data.

You can fabricate the sample data yourself once you understand what's realistic. We will ask why your sample data looks the way it does.

The three sources

1. SAP, fuel and procurement data

SAP exports are not friendly. Research what a typical export looks like (IDoc, flat file, OData service, BAPI, pick one and justify). Expect inconsistent units, plant codes that mean nothing without a lookup table, German column headers in some configurations, and dates in formats you don't want. Decide what subset of SAP reality you're handling.

2. Utility data, electricity

However a facilities team typically gets this: a portal CSV export, a PDF bill, an API if the utility offers one. Pick one mode and justify the choice. Account for the fact that meter readings have units, tariff structures, and billing periods that don't align with calendar months.

3. Corporate travel, flights, hotels, ground transport

Look at how platforms like Concur, Navan, or similar expose this. Different categories imply different emission factors. Distances aren't always given; sometimes you only get airport codes.

For each source, you decide the ingestion mechanism (file upload, API pull, manual paste, whatever fits the realistic shape). Justify it.

Deliverables

1. **Working app, deployed.** Deployment is mandatory. Render, Railway, Fly, or any provider of your choice. Local-only submissions will not be reviewed. Share the live URL in your submission email.
2. **MODEL.md.** Your data model and why. Must handle multi-tenancy, Scope 1/2/3 categorization, source-of-truth tracking (which source produced this row, when, was it edited), unit normalization, and audit trail. This document carries significant weight in evaluation.
3. **DECISIONS.md.** Every ambiguity you resolved, what you chose, why, and what you'd ask the PM if you could. Include what subset of each source you chose to handle and what you ignored.
4. **TRADEOFFS.md.** Three things you deliberately did *not* build and why.
5. **SOURCES.md.** For each of the three sources: what real-world format you researched, what you learned, what your sample data looks like and why, and what would break in a real deployment.

Submission and access

Reply to the email this assignment was sent from with a link to your GitHub repository, a link to your deployed app, and any credentials needed to log in. Share your repository (private is fine) with the following accounts:

- saurav@breatheesg.com
- rahul@breatheesg.com
- shivang@breatheesg.com

How we grade

- **35%** data model quality
- **25%** defense of your decisions in the post-submission review
- **20%** how realistically you handled each source (research showing in your choices)
- **10%** analyst UX (can a non-engineer use this?)
- **10%** what you chose not to build

Good luck. We're looking forward to seeing how you think.